The DFE chain, one stage per ELF

These four kernels are the DFE's signal path, each built as a standalone program so a host can push arrays through one stage at a time. That is what the link-level co-simulation (`cosim/scripts/dfe_cosim.py`) needs: run TX, put a channel in the middle, run RX, and count bit errors.

kernel	in → out	what it is
`compute_dfe_tx`	61 subcarriers → 1024 samples	reorder → iFFT-1024 → bit-reversal → windowing
`compute_x2_interp`	1088 → 2176 samples	2× polyphase FIR interpolator, 32 taps
`compute_decim_2x_8tap`	2176 → 1088 samples	2× FIR decimator, 8 taps
`compute_dfe_rx`	1024 samples → 61 subcarriers	FFT-1024 → bit-reversal → subcarrier extraction

Nothing here is a copy of NXP's DSP. Every kernel links against `firmware/lib/lib_DFE_Ref_vspa2.elb` — the *same* library the firmware links — so the co-simulation exercises the real iFFT, FFT and filters. The `.c` files are only harnesses: they set up buffers and call in.

```
make kernels                  # -> build/compute_*.eld
python3 cosim/scripts/kernel_check.py  # each kernel vs an independent Python model
python3 cosim/scripts/dfe_cosim.py     # BER vs SNR through the whole chain
```

Two things you must get right

The rate converters process whole 64-sample blocks

`X2_interp_tap32_filter`'s own header says it: "*number of input samples must be a multiple of 64 samples.*" Feed it a non-multiple and it does not error — it processes the whole blocks and leaves the tail as **garbage**.

This is not hypothetical. The original co-simulation used a 20-sample cyclic prefix, so it fed the interpolator 1024 + 20 = **1044** samples. 1044 = 16×64 + 20. Measured: output samples 0–2047 were bit-exact and samples **2048–2087 were garbage** — and those corrupted samples land in the cyclic prefix, which is exactly the region the receiver slices. It still produced a plausible BER curve.

So the cyclic prefix here is **64 samples**: 1024 + 64 = 1088 = 17×64. ✓

The interpolator's taps are stored in reverse *pair* order

The hardware wants `h[M-2], h[M-1], h[M-4], h[M-3], …, h[0], h[1]`. A Python model wants natural order. Converting between them is **not** a no-op even though the filter is a palindrome — it reverses the order of the *pairs*, not of the taps. Get it wrong and the filter is quietly a different filter (measured: 2 dB SNR instead of bit-exact).

How data gets in and out

It is not read from a file. The host writes the input buffer and reads the output buffer **directly out of DMEM, by ELF symbol** — see `cosim/dfe/kernel_runner.py`.

Two reasons. First, the simulator runs *in this process*, so its memory is simply there; files were only ever a way to talk to a separate `runsim` process. Second, the native ISS services exactly three syscalls — `write`, `clock`, `time`. **`open` and `read` return −1**, so a harness that `fopen`s its vectors gets a NULL `FILE*`.

One subtlety, because it is silent when you get it wrong: the buffers are C globals, so they live in `.bss`, and crt0 **zeroes `.bss` at boot** — after the image is loaded. A host write between `load_elf` and `boot` is therefore wiped, and the kernel computes on zeros and returns a clean, plausible, wrong answer. The runner writes its inputs at a **breakpoint on `main`**: past crt0, before the kernel has looked at them.